Strengths and Limitations of Apache Kafka as a Data Platform Technology

Name: Luna Sugiyama **Student ID:** 48-256422

1. Selected Technology

Apache Kafka is a distributed data streaming platform designed to handle large volumes of event data in near real time. It was originally developed as a distributed messaging system for log processing, but it is now widely used as a data platform for event-driven systems, monitoring, IoT, financial transactions, and machine-learning pipelines. Kafka follows a publish/subscribe model in which producers write records to topics and consumers read those records asynchronously. Topics can be divided into partitions and distributed across brokers, allowing Kafka to support parallel data ingestion and consumption [1, 2].

2. Strengths

The main strength of Kafka is scalability. Since topics are divided into partitions, multiple brokers and consumers can process different partitions in parallel. This enables high-throughput data streaming and makes Kafka suitable for systems that continuously generate large amounts of data. Consumer groups also distribute partitions among consumers, providing load balancing and allowing processing capacity to be increased by adding more consumers [2].

Another strength is fault tolerance. Kafka replicates partitions across brokers, so if one broker fails, another replica can continue serving the data. This design reduces the risk of data loss and improves availability. Kafka also stores event records as durable logs, which allows consumers to replay previous events. Replayability is valuable because the same raw data can be reused for debugging, recovery, testing, or the creation of new analytics pipelines.

Kafka also helps decouple systems. Producers do not need to know which applications will consume their data, and consumers can be added or removed without changing the producer side. This makes Kafka useful as a central data backbone when many applications share the same data streams. It can also integrate with databases, data lakes, stream-processing frameworks, and machine-learning systems, making it suitable for heterogeneous data platforms [2].

A further advantage is that Kafka supports real-time decision making. Distributed stream processing is important because data such as fraud alerts, sensor readings, patient monitoring data, and financial transactions can rapidly lose value if they are not processed immediately [3]. Kafka enables this data to be delivered continuously rather than waiting for periodic batch processing.

3. Limitations

Kafka also has important limitations. First, it is operationally complex. A production cluster requires careful design of brokers, partitions, replication factors, retention policies, consumer groups, monitoring, and security. Performance depends strongly on configuration choices such as message size, serialization, batching, compression, and partitioning [2]. Poor configuration can create hot spots, uneven workloads, high latency, or wasted resources.

Second, Kafka does not automatically solve all data consistency problems. Ordering is guaranteed only within a partition, not across an entire topic. Developers must also handle duplicate events, schema evolution, consumer offsets, failed consumers, and exactly-once processing assumptions. These issues make application development more difficult.

Third, consumer lag can become a serious problem. If producers generate data faster than consumers can process it, unprocessed records accumulate. This increases latency and may eventually exceed the configured retention period. Operators must therefore monitor throughput, lag, disk usage, and broker health continuously.

Kafka is also not a general-purpose database. It is optimized for sequential event logs rather than arbitrary queries, relational joins, or interactive analysis over historical data. It usually needs to be combined with databases, search engines, data warehouses, or data lakes, increasing architectural complexity and cost.

Storage and network costs are another limitation. Replication improves fault tolerance, but storing several copies of each partition consumes additional disk space and network bandwidth. Long retention periods can further increase infrastructure requirements.

Finally, security requires careful configuration. Authentication, authorization, encryption, access control, and secret management must be set up correctly, particularly when Kafka connects many systems across an organization. For small applications with low traffic, this operational burden may outweigh Kafka's advantages, and a simpler queue or managed service may be more appropriate.

4. Conclusion

Apache Kafka is a powerful data platform technology when an organization needs scalable, durable, fault-tolerant, and real-time event streaming. Its greatest value is that it converts continuously generated data into reusable streams that can be shared by many applications. However, this value comes with configuration, monitoring, storage, security, and architectural complexity. Kafka should therefore be selected when its scalability, replayability, and real-time integration benefits justify the cost of operating a distributed streaming platform.

References

[1] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB*, 2011.

[2] T. P. Raptis and A. Passarella, "A survey on networked data streaming with Apache Kafka," *IEEE Access*, vol. 11, pp. 85333–85350, 2023, doi: 10.1109/ACCESS.2023.3303810.

[3] H. Isah, T. Abughofa, S. Mahfuz, D. Ajerla, F. Zulkernine, and S. Khan, "A survey of distributed data stream processing frameworks," *IEEE Access*, vol. 7, pp. 154300–154316, 2019, doi: 10.1109/ACCESS.2019.2946884.